

Leetcode 35 :- Search Insert Position →

↳ $O(n)$

([1, 3, 5, 6], target = 2)
↑ ↑ ↘ ↘
i=0 i=1 i=2 i=3 target = 4
 target = 7

(10)

(10)

1 <= 2 → ans = i + 1

~~3 <= 2 ✗~~

for (i = 0; i < n; i++) {

if (nums[i] >= target)
return i;

else if (nums[i] < target)
{ ans = i + 1;

}
return ans;

if (nums[i] == target)
return i;

return nums.length;

$[1, 3, 5, 7]$
 $\begin{matrix} \uparrow & \uparrow & \uparrow & \uparrow \\ 0 & 1 & 2 & 3 \end{matrix}$
 start mid start end
 X

$[5, 7]$
 $\begin{matrix} \uparrow & \uparrow & \uparrow \\ 1 & 2 & 3 \end{matrix}$
 end mid start

target = 4

return start

100

$[1, 2, 3, 4, 5]$
 $\begin{matrix} \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ 0 & 1 & 2 & 3 & 4 \end{matrix}$
 mid start

$(start = 4 + 1)$
 $= 5$

$[10, 20, 30, 40, 50]$, target = 30, 45, 60

0 1 2 3 4
↑ ↑ ↑ ↑ ↑
s e

Start = 0, end = nums.length - 1;

$$\left(\frac{\text{Start} + \text{end}}{2} \right)$$

```
while (start <= end) {  
    int mid = (start + end) / 2;  
    if (nums[mid] == target)  
        return mid;  
    else if (nums[mid] > target)  
        end = mid - 1;  
}
```

```
else start = mid + 1;  
}  
return start;
```

```

size = nums.length;
int first = firstOccurrence (nums, size, target);
int last =

```

```

return new int[] { first, last };

```

[2, 3, 3, 5, 5, 5, 5, 5, 5, 7, 7, 7, 7, 8, 8]
 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

↑ start ↑ mid ↑ end
 → ans = mid
 end = mid - 1

5 5 5 5 5
 ↑

1 2

```

int[] arr = { 1, 2, 3 };
int arr;
arr = new int[] { 1, 2, 3 };

```

```

for (i = 0 -> n) {
  if (arr[i] == target)
    last = i;
}

```

```

int[] arr = { first, last };
return arr;

```

```

public int firstOccurrence (int[] nums, int start, int target) {
  ...
  return first;
}

```


$(2, 2, 3, 3, 3, 3, 3, 3, 3, 4, 5, 5, 5)$
 0 1 2 3 4 5 6 7 8 9 10 11 12
 ↑ ↑ ↑ ↑ ↑
 s end mid e ~~mid~~

0 1
 ↑ ↑
 mid s
 e m

```

if (arr[mid] == target) {
  ans = mid;
  end = mid - 1;
}
else if (arr[mid] > target) {
  end = mid - 1;
}
else {
  start = mid + 1;
}

```

ans = 6



</> Code

Java ▾ 🔒 Auto



```
1  import java.util.*;
2
3  class Solution {
4      public int firstOccurrence(int []nums,int size,int target){
5          int start = 0, end = size - 1;
6          int ans = -1;
7
8          while(start <= end){
9              int mid = (start + end) / 2;
10             if(nums[mid] == target ){
11                 ans = mid;
12                 end = mid - 1;
13             }else if(nums[mid] > target) end = mid - 1;
14
15             else start = mid + 1;
16         }
17         return ans;
18     }
19
20
21     public int lastOccurrence(int []nums,int size,int target){
22         int start = 0, end = size - 1;
23         int ans = -1;
24
25         while(start <= end){
26             int mid = (start + end) / 2;
27             if(nums[mid] == target ){
28                 ans = mid;
29                 start = mid + 1;
30             }else if(nums[mid] > target) end = mid - 1;
31
32             else start = mid + 1;
33         }
34         return ans;
35     }
36
37     public int[] searchRange(int[] nums, int target) {
38         int size = nums.length;
```